

Boosting and Additive Models

Marco Colangelo¹

¹Department of Computer Science, University of Milan, Milan, Italy
Corresponding author: marco.colangelo@studenti.unimi.it

September 14, 2026

Abstract

!!!!!!

1 Introduction

1.1 From a single predictor to an ensemble

Supervised learning algorithms map an annotated training set to a predictor $f : X \rightarrow Y$, and the empirical risk minimization (ERM) selects, within a fixed hypothesis class, the predictor with smallest training error. The statistical analysis of ERM makes the tension inherent in this choice explicit: for a finite class $\mathcal{H}$, the estimation error of the minimizer is controlled by a term of order $\sqrt{\ln |\mathcal{H}|/m}$, so shrinking $\mathcal{H}$ improves the control on the variance but may increase the approximation error, and enlarging it does the opposite. A predictor is thus always a compromise struck once, before seeing how it fails.

Ensemble methods sidestep this farming. Rather than searching harder within a class, they combine many predictors from a deliberately impoverished class into a single, more expressive one. The simplest instance is the majority classifier $f(x) = \text{sgn}(\sum_{t=1}^T h_t(x))$: if the base classifiers made *independent* mistakes with respect to the uniform distribution over the training set, a Chernoff-Hoeffding argument shows that the training error of f decays as $e^{-2T\gamma^2}$, where γ measures how much better than random guessing h_t is. Independence, however, is a strong and largely unverifiable assumption. Bagging attempts to approximate it by resampling; the contribution of boosting is to obtain the same exponential decay *without* requiring it.

1.2 Boosting and Additive models

Boosting builds an ensemble incrementally. At each round it presents the base learner with a reweighted version of the training set, concentrating probability mass on the examples that the current ensemble finds hard, and appends the resulting base classifier to a weighted sum

$$f = \sum_{t=1}^T w_t h_t, \quad (1)$$

Equation (1) is the reason the two halves of this project's title belong together: AdaBoost is not only a device for amplifying weak learners, it is also a procedure that fits an *additive model* in a dictionary of simple functions, one term at a time, never revisiting the coefficients already chosen. This second reading, made explicit by Friedman, Hastie and Tibshirani, is what connects boosting to surrogate losses: the coefficients w_t are exactly those that greedily minimise an exponential upper bound on the zero-one loss.

1.3 Objectives of the study

The aim of this work is to implement AdaBoost with decision stumps from scratch and to study empirically what its theoretical guarantees do and do not predict.

1. **How tight is the training-error bound?** The exponential bound $\prod_t \sqrt{4\varepsilon_t(1-\varepsilon_t)}$ is a worst-case guarantee. We track it round by round against the observed training error.
2. **How does the weak learning assumption degrade?** The bound is driven by the edges $\gamma_t = \frac{1}{2} - \varepsilon_t$, and there is no reason for these to stay bounded away from zero as boosting proceeds. We measure their decay directly.

3. **What happens to the test error once the training error is zero?** Classical model-selection reasoning predicts overfitting as T grows; the boosting literature repeatedly reports the opposite. We examine both regimes.
4. **How does label noise change the picture?** The exponential loss grows without bound on misclassified point, which suggests that corrupted labels should attract a disproportionate share of the distribution. This intuition is made precise by the negative result of Long and Servedio; we test it with a controlled flip rate on labels.

The comparison against a single decision tree and logistic regression, required by the assignment, is used to place these observations in context rather than as an end in itself: the interesting outcome is not which algorithm wins, but on *which* dataset each of them wins and why.

2 Theoretical Framework

3 Results

4 Conclusion